

D4 — AI Usage Journal — Part 1

CSE 341 — Concepts of Programming Languages
Gebze Technical University

Language: WorkflowScript

Student: Ömer Faruk Koç

Student #: 210104004061

Part: 1

Entries: 10

Submitted: 2026-05-08

Journal Overview

Entry	Date	Phase	Topic
1	2026-05-06	Design	DSL domain selection
2	2026-05-06	Design	Language-runtime independence
3	2026-05-06	Design	Domain reconsideration — RuleScript to WorkflowScript
4	2026-05-06	Design	Core language design decisions (6 decisions)
5	2026-05-06	Design/Grammar	EBNF grammar writing + Gemini critique evaluation
6	2026-05-07	Design	Lexical structure — boolean literal classification
7	2026-05-07	Design	Names, binding, scope, lifetime
8	2026-05-08	Testing	TDD test suites (lexer, parser, checker, interpreter)
9	2026-05-07	Writing	Example programs + smoke testing
10 (E1)	2026-05-08	Design	EBNF grammar generation — Experiment E1

[illegible]

Figure 1: Resim Açıklaması Buraya Yazılır

Entry 1 — DSL Domain Selection

Field	Content
Entry #	1 of 10
Date	2026-05-06
Phase	Design
AI tool	Claude Sonnet 4.6
Goal	Choosing a suitable DSL domain for the project.
Prompt	“backend alanıyla ilgileniyorum bu alandaki bir boşluğu dolduracak işleri kolaylaştıracak bir dsl olsun. Ayrıca golang de compiler için kütüphanelerin olduğunu biliyorum. Daha önce parser falan yazmıştım. Dil olarak go kullanmayı düşünüyorum o yüzden.”
Response (key part)	RuleScript domain proposal — backend business rule engine concept
Accepted	RuleScript domain proposal accepted.
Rejected / modified	VectorLang, RobotScript domain proposal — its answer to the question “why a new language for backend business rule engine” is weaker
Errors I caught	—
Reflection	<i>AI analyzed why the domain was chosen as a DSL rather than a general scripting language when offering domain options.</i>

Entry 2 — Language–Runtime Independence

Field	Content
Entry #	2 of 10
Date	2026-05-06
Phase	Design
AI tool	Claude Sonnet 4.6
Goal	Verify whether RuleScript is tied to Go or can work independently.
Prompt	“Rulescript sadece Go ile mi çalışabilir olmayacak değil mi? Bu DSL her dille çalışabiliyor olması lazım.”
Response	The AI clarified that the interpreter being written in Go does not make the language itself Go-specific. It drew analogies: Lua (written in C, embeddable in Go, Java, Python, etc.), SQL (each database writes its own parser/runtime, the language standard is independent), Terraform HCL (written in Go, runs on AWS, Azure, GCP). The conclusion was: RuleScript as a language is independent. The Go implementation is one runtime. Any backend (Python, Java, Node.js) could have its own RuleScript interpreter.
Accepted	The language/runtime separation concept. This actually strengthens the domain justification — RuleScript is not a Go-specific tool, it is a language that any backend could adopt. The Go interpreter is the reference implementation.
Rejected / modified	Nothing rejected.
Errors I caught	None in this exchange.
Reflection	<i>I didn't realize that SQL is actually a DSL . I used to think of it just as a database tool, but the AI clarified that since it focuses solely on managing relational data rather than being a general-purpose language. This helped me to better understand the architectural role of DSL.</i>

Entry 3 — Domain Reconsideration: RuleScript to WorkflowScript

Field	Content
Entry #	3 of 10
Date	2026-05-06
Phase	Design
AI tool	Claude Sonnet 4.6
Goal	Reconsider domain choice — RuleScript felt out of place next to the handout’s example languages.
Prompt	“tekrar düşündüm de Rulescript dokümanda verilen örneklere göre biraz soyut kahyor. Başka bir tavsiye ver yine aynı domain olsun”
Response	The AI identified a structural pattern across all handout examples: each one follows “describe domain → produce concrete output” (TurtleLang draws, CircuitLang simulates, RecipeLang scales). RuleScript broke this pattern by being too abstract/enterprise-feeling. Three alternatives were proposed: NetSim (network topology simulation), MigrateScript (database migration language, produces SQL), WorkflowScript (typed task/pipeline orchestration, like a Makefile with control flow and a type system). The “why a new language” justification for WorkflowScript: “YAML/Makefile configs are untyped — errors surface at runtime, not at parse time. WorkflowScript catches them earlier and supports real control flow.”
Accepted	WorkflowScript as the domain. The “describe → execute → observe output” pattern matches the handout examples. The justification for a new language is concrete and defensible: type safety at parse time vs. silent YAML failures at runtime.
Rejected / modified	NetSim and MigrateScript were set aside. NetSim felt too close to CircuitLang (already in the handout). MigrateScript is heavily tied to SQL output, which would complicate the interpreter scope for Part 1.
Errors I caught	None. The pattern analysis was sound and the alternatives were genuinely distinct from the handout list.
Reflection	<i>Rulescript was very abstract, and the AI explanations didn’t satisfy me. I didn’t want to proceed further without being sure. That’s why I asked for advice again.</i>

Entry 4 — Core Language Design Decisions

Field	Content
Entry #	4 of 10
Date	2026-05-06
Phase	Design
AI tool	Claude Sonnet 4.6
Goal	Make all 6 core language design decisions for WorkflowScript before writing the EBNF grammar.
Prompt	“onaylıyorum” (repeated across 6 decision points). Earlier prompts in the same session: “Karar 1 A seçeneği olsun diğerlerini göster”, “B seçeneği ve C yi birleştirek nasıl olur?”, “vazgeçtim sadece B olsun”, “C neden olmamalı sence?”
Response	The AI presented options for each decision with exam defense reasoning attached. Key arguments: Name equivalence for task (“Two tasks with identical fields still represent different domain entities — conflating them would be a semantic error.”), Static scoping (“You can determine which value is bound to a variable by reading the code, without running it.”), int→float coercion in arithmetic only (“Implicit narrowing causes data loss — only widening is safe.”), pipeline vs function (“A function computes a value; a pipeline declaratively defines an execution order.”).
Accepted	All 6 decisions as confirmed: (1) Primitive types: int, float, bool. (2) Structured type: task record (name equivalence). (3) Control structures: if/else, while. (4) Domain-specific: task{ }, pipeline{stage}, run, on_failure{ }. (5) Scoping: static (lexical). (6) Expression: standard precedence, left-associative, int→float widening in arithmetic only.
Rejected / modified	when success/failure construct was initially accepted then dropped in favor of simpler if/else + while. Reason: — minimizing parser complexity without weakening the domain-specific justification. duration as a 4th primitive type was proposed but rejected — adds lexer complexity with marginal benefit for Part 1 scope.
Errors I caught	The AI initially recommended option B over C for the domain-specific construct without a strong reason. When questioned (“C neden olmamalı sence?”), it admitted the only reason was deadline pressure and that C was actually the stronger academic choice. The recommendation changed after pushback. This was a case of the AI being overly conservative rather than wrong on substance.
Reflection	<i>I used AI to analyze the good and bad cases of the 6 decisions required for EBNF grammar design, thus avoiding any assumptions.</i>

Entry 5 — EBNF Grammar Evaluation Critique

Field	Content
Entry #	5 of 10
Date	2026-05-06
Phase	Design / Grammar
AI tool	Claude Sonnet 4.6 , Gemini
Goal	Write the EBNF grammar for WorkflowScript and critically evaluate an AI critique of that grammar.
Prompt	“Gemini EBNF için aldığımız kararlara şöyle diyor: 1. Task’ların ‘Gövdesi’ (Body) Yok [...] 2. Kritik Veri Tipleri Eksik (string ve void) [...] 3. run İfadesinin Değer Döndürememesi [...] 4. <field_expr> Kullanım Alanı Belirsiz [...] 5. Pipeline Aşama Geçişleri [...]”
Response	Claude evaluated each of Gemini’s five points and recommended accepting only one: adding <code>void</code> as a return type. The rest were rejected with explicit reasoning. The user’s decisions remained unchanged “WorkflowScript tanımlar, execute etmez — bu separation of concerns için gerekli.” and “Zaten field access var: run build, sonra build.timeout ile state’e erişilir. Expression olması gerekmez.”
Accepted	<code>void</code> return type. Functions that perform side effects (like notifying, logging) should not be forced to return a value. This is a small grammar addition with clear semantic value. Added: <code><type> ::= "int" "float" "bool" "task" "void"</code>
Rejected / modified	action field + string type: Would require string literals in the lexer and conflicts with WorkflowScript’s design as a simulation/description DSL, not a command executor. run as expression: Unnecessary — task state is accessible via field access (<code>build.timeout</code> , <code>build.retries</code>) after run. Stage parameters: Adds complexity without adding expressiveness — pipelines reference tasks, not command strings.
Errors I caught	Gemini assumed WorkflowScript would execute shell commands (hence suggesting action: <code>"go build ./..."</code>). — tasks describe behavior, not commands.
Reflection	<i>Gemini did not ask about the design intent before critiquing. This shows AI feedback can be technically correct but contextually wrong if it doesn't know your design goals.</i>

Entry 6 — Lexical Structure: Boolean Literal Classification

Field	Content
Entry #	6 of 10
Date	2026-05-07
Phase	Design — Lexical Structure
AI tool	Claude Sonnet 4.6
Goal	Determine token categories and decide how boolean literals should be classified (keyword vs. literal).
Prompt	“true ve false AST olarak literal olmalı, fakat Lexerda keyword olarak tanımlanmalı. Bir de sence string eklemeli miyiz dilin konseptini düşünürsek?”
Response	The AI confirmed user’s point and clarified the two-level distinction: Lexer level: “true” is in the keyword list → produces BOOL_LIT token (reserved word). AST level: represented as BoolLiteral(value: true), just like 42 → IntLiteral(value: 42). The AI also mapped the token pipeline: “true” → BOOL_LIT → BoolLiteral(value: true), “42” → INT_LIT → IntLiteral(value: 42), “build” → IDENT → Identifier(name: “build”).
Accepted	The two-level classification of true/false. Treating them as reserved keywords in the lexer prevents their use as variable names, while representing them as literals in the AST gives them the correct semantic meaning.
Rejected / modified	AI’s suggestion to add a string type was rejected. Adding string literals would complicate the lexer without adding domain value — the DSL abstracts away command strings by design.
Errors I caught	None. The token pipeline the AI described is consistent with what I independently expected.
Reflection	<i>We were able to solve true-false concept by distinguishing between lexer and AST.</i>

Entry 7 — Names, Binding, Scope, Lifetime

Field	Content
Entry #	7 of 10
Date	2026-05-07
Phase	Design — Names, Binding, Scope, Lifetime
AI tool	Claude Sonnet 4.6
Goal	Establish scoping rules, shadowing policy, and lifetime categories using Sebesta Ch. 5 terminology.
Prompt	“Names, Binding, Scope, Lifetime’ı Sebesta terminolojisiyle belirle, ben onaylayacağım veya değiştireceğim.”
Response	The AI produced a lifetime table mapping each language construct to a Sebesta category: task / pipeline declarations → Static lifetime, Global var → Static lifetime, Function parameters → Stack-dynamic lifetime, Local var inside a block → Stack-dynamic lifetime. It also asked whether shadowing should be allowed, noting that Go permits it and that it is the natural implementation path when using a linked-list scope chain.
Accepted	The lifetime table as written. The separation of static lifetime (task, pipeline, global var) from stack-dynamic (locals, parameters) is consistent with the decision that task declarations are top-level only — a task cannot be declared inside a function precisely because its lifetime must be static. Shadowing allowed (Option A). Justified because inner scopes creating their own bindings without invalidating outer ones is standard lexical scoping behaviour. Forcing a compile error on shadow would add complexity to the checker with little safety gain for this domain.
Rejected / modified	Nothing rejected.
Errors I caught	None. The table matched my own reasoning.
Reflection	<i>The decision for task declarations to have a static lifetime was previously made for pipeline references. Therefore, I simply accepted it and continue.</i>

Entry 8 — TDD Test Suites

Field	Content
Entry #	8 of 10
Date	2026-05-08
Phase	Testing
AI tool	Claude Sonnet 4.6
Goal	Write test suites for the lexer, parser, and type checker following TDD — failing tests first, then implementation.
Prompt	“phase-1 den implementasyona başla” (followed by “devam et, eğer sorun yoksa” to advance each phase). Final effective prompt for each phase: “Önce başarısız olan testleri yaz. Başarısız olduklarımı doğrulamak için yeniden çalıştır. Ardından implement et hepsi başarılı olmalı.”
Response	The AI generated four test files, each following the same helper-function pattern: Lexer (6 tests: TestLexer_BasicTokens, TestLexer_LineNumbers, TestLexer_IllegalChar), Parser (8 tests including TestParser_BinaryExpr_Precedence verifying right subtree is BinaryExpr(*) for <code>var x: int = 3 + 4 * 2</code>), Type checker (8 tests including TestChecker_IntToFloatCoercion for <code>var x: float = 1 + 2.5</code>), Interpreter (6 tests including TestEval_StaticScoping verifying <code>y == 10</code>). All 28 tests passed across all 4 packages.
Accepted	The shared helper pattern (parse / check / run) that wraps the full pipeline in one line — each test reads as plain English source code rather than token manipulation. The precedence test ($3 + 4 * 2 \rightarrow \text{left: } 3, \text{right: } 4*2$) and the static-scoping test were kept exactly: they verify the two most failure-prone invariants in the design.
Rejected / modified	None of the generated tests were rejected. The plan pre-specified the exact test code, so the AI’s role was faithful transcription rather than independent test design. A more honest evaluation would have asked the AI to write tests from the language spec without seeing the implementation — which would have been a stronger test of AI judgement.
Errors I caught	None caught within the test files themselves. The tests were generated from the plan and matched the implementation spec exactly, so there was no divergence to detect. The real error-catching happened downstream — in Phase 7 smoke tests (see Entry 9).
Reflection	<i>The implementation plan included TDD. Therefore, AI performed the tests without knowing the implementation details. This increased the reliability of the tests.</i>

Entry 9 — Example Programs + Smoke Testing

Field	Content
Entry #	9 of 10
Date	2026-05-08
Phase	Writing
AI tool	Claude Sonnet 4.6
Goal	Write three valid and five invalid WorkflowScript example programs that exercise all language features and produce clear, line-numbered error messages.
Prompt	“kodların çalıştığını doğrulamak için example programlar yazmamız gerekiyor. Bazıları valid olsun çalıştığını görelim. Bazıları da invalid olsun. parser hata verip uygun hata mesajları vermesi gerekiyor.”
Response	The AI produced all 8 example files and README.md. The most complete valid program (03_pipeline.ws) included task definitions (build, test, deploy), a pipeline with stages and on_failure, a function should_run, and a while loop with if/run. Invalid program errors (all line-numbered, exit code 1): 02_bad_field_order.ws → “line 1: expected KW_TIMEOUT, got retries”, 03_empty_pipeline.ws → “line 1: pipeline must have at least one stage”, 04_missing_arrow.ws → “line 1: expected ARROW, got int”, 05_bad_parallel.ws → “line 1: expected BOOL_LIT, got 1”.
Accepted	All 8 example files were kept. The invalid programs cover exactly one error each and produce the expected message format — they serve as both human-readable documentation and implicit regression tests. The README was accepted verbatim; it is minimal by design (a DSL tutorial, not a Go package readme).
Rejected / modified	The parser’s field-access rule had to be patched. The plan specified <code>p.expect(token.IDENT)</code> after a dot, but <code>timeout</code> , <code>retries</code> , and <code>parallel</code> lex as keyword tokens (KW_TIMEOUT etc.), not IDENT. Running <code>02_functions.ws</code> (which contains <code>t.timeout</code>) exposed the bug. The fix accepted keyword tokens as valid field names after a dot. This change was not in the original plan.
Errors I caught	The KW_TIMEOUT field-access bug. Detection method: the smoke test <code>./wsc -dump-ast examples/valid/02_functions.ws</code> failed with a parse error on <code>t.timeout</code> . Without the example programs as an integration test, the unit tests would have passed and the bug would have remained hidden — no parser test used dot-access on a keyword-named field. This shows that unit tests alone do not cover integration paths; end-to-end examples caught what 28 unit tests missed.
Reflection	<i>These legal and illegal programs have clearly shown us the limits of the language we initially defined.</i>

Entry 10 — Experiment E1: EBNF Grammar Generation

Experiment E1 task (from §6): “Ask the AI to generate an EBNF grammar for your language from a one-paragraph description. Compare its grammar with yours. What did it get wrong or suboptimal? Does it handle operator precedence correctly?”

Field	Content
Entry #	10 of 10
Date	2026-05-08
Phase	Design
AI tool	Claude Opus 4.6
Goal	Generate an EBNF grammar for WorkflowScript from a one-paragraph description and compare with my own.
Prompt	<i>"Sana bir DSL description vereceğim. Buna göre bana tam bir EBNF grammar oluştur. Başka soru sorma. Bazi spekulasyonlar olacağının farkındayım."</i>
Response	A large grammar with generics, <code>import/const/type</code> declarations, arbitrary task fields, <code>workflow</code> (not <code>pipeline</code>), <code>triggers</code> , <code>try/catch</code> , <code>let/mut</code> with optional type annotations, lambda expressions, <code>duration</code> literals, and <code>run</code> taking a shell string literal. Operator precedence: <code> </code> → <code>&&</code> → equality → comparison → additive → multiplicative → unary, one rule per level.
Accepted	The precedence hierarchy matches our design exactly, confirming it is standard. The <code>on_failure</code> block was independently produced, validating that design decision.
Rejected / modified	Rejected: imports, generics, <code>string/duration/null</code> types, arbitrary task field keys, <code>let/mut</code> , optional type annotations, <code>try/catch</code> , lambdas, ternary, compound assignment, postfix <code>!</code> . None of these are in our design. Renamed <code>workflow</code> back to <code>pipeline</code> .
Errors I caught	(1) <code>run</code> takes a shell string (<code>run "go build"</code>) instead of a task identifier (<code>run build</code>). This would destroy the named-task abstraction. (2) Optional type annotation (<code>let x = 10</code>) silently requires type inference our type checker does not implement. (3) Task field catch-all (<code> identifier</code>) accepts <code>task build { banana: 42 }</code> as valid. All three caught by comparing against our documented design decisions.
Reflection	<i>Operator precedence was handled correctly — the standard one-rule-per-level structure matched ours. However, the AI defaulted to general-purpose language features (TypeScript/Rust style) rather than inferring domain constraints. The <code>run</code> error was the most dangerous: without knowing our design intent, it looks plausible. Lesson: state domain-specific constraints explicitly in the prompt.</i>